

SmartFête - Architecture Documentation

Table of Contents

- [1. Executive Summary](#)
- [2. System Overview](#)
- [3. Technology Stack](#)
- [4. Architecture Patterns](#)
- [5. Component Architecture](#)
- [6. API Architecture](#)
- [7. Data Flow](#)
- [8. Affiliate System](#)
- [9. Security Considerations](#)
- [10. Deployment Architecture](#)
- [11. Performance Optimization](#)
- [12. Future Roadmap](#)

Executive Summary

SmartFête is an AI-powered shopping assistant website that helps users find the best deals from multiple stores while staying within their budget. The application features international currency support, location-based product recommendations, and a comprehensive affiliate link system with commission tracking.

Key Features

- Multi-Store Product Search:** Amazon, eBay, Walmart, Target integration
- AI Shopping Assistant:** Intelligent product recommendations
- Multi-Currency Support:** Real-time currency conversion
- Affiliate Commission System:** Comprehensive link tracking and analytics
- Budget Optimization:** Alternative suggestions and price comparisons
- Location-Based Recommendations:** Geo-targeted product suggestions
- Responsive International Design:** Mobile-first approach

System Overview

SmartFête is built as a modern web application using Next.js 15 with TypeScript, implementing a serverless architecture pattern with API routes and React Server Components.

Core Architecture Principles

- Component-Based Architecture:** Modular React components
- API-First Design:** RESTful API endpoints
- Serverless Functions:** Next.js API routes
- Responsive Design:** Mobile-first Tailwind CSS
- Type Safety:** Full TypeScript implementation

System Boundaries

SmartFête System	
Frontend (Next.js 15 + React 19)	
├── Components Layer	
├── API Integration Layer	
└── UI/UX Layer (Tailwind CSS)	
Backend API Layer (Next.js API Routes)	
├── Search API	
├── Currency API	
├── Analytics API	
├── Usage Tracking API	
└── Payment API	
External Integrations	
├── Amazon Product Advertising API	
├── eBay Browse API	
├── Walmart Open API	
├── Target Redsky API	
├── Currency Exchange APIs	
└── Stripe Payment Processing	

Technology Stack

Frontend Technologies

- **Next.js 15.4.4:** React framework with App Router
- **React 19.1.0:** User interface library
- **TypeScript 5:** Type-safe JavaScript
- **Tailwind CSS 4:** Utility-first CSS framework
- **Lucide React 0.525.0:** Icon library

Backend & API

- **Next.js API Routes:** Serverless functions
- **Axios 1.11.0:** HTTP client for external APIs
- **Cheerio 1.1.2:** Server-side HTML parsing
- **Puppeteer 24.15.0:** Web scraping capabilities

External Services

- **Stripe 18.3.0:** Payment processing
- **Clerk Next.js 6.27.1:** Authentication (configured)
- **TanStack React Query 5.83.0:** Data fetching and caching

Development Tools

- **ESLint 9**: Code linting
 - **PostCSS**: CSS processing
 - **Currency Codes 2.2.0**: International currency support
-

Architecture Patterns

1. Component Architecture Pattern

SmartFête follows a hierarchical component structure:

```
App Layout (layout.tsx)
├── Main Page (page.tsx)
│   ├── BudgetInput Component
│   ├── CurrencySelector Component
│   ├── ProductResults Component
│   ├── ShoppingList Component
│   ├── Wishlist Component
│   ├── UsageQuota Component
│   ├── AffiliateProduct Component
│   └── AffiliateAnalytics Component
```

2. API Route Pattern

RESTful API design with clear separation of concerns:

```
/api/
├── search/      # Product search across stores
├── currency/     # Currency conversion
├── usage/       # Usage quota tracking
├── payments/    # Payment processing
└── analytics/
    └── affiliate-click/ # Affiliate click tracking
```

3. Service Layer Pattern

Core business logic abstracted into service modules:

```
/lib/
├── api-integrator.ts # External API integration
├── affiliate-manager.ts # Affiliate link management
└── [future services]
```

Component Architecture

Core Components

1. BudgetInput Component

Purpose: User budget input and validation

Dependencies: Currency selector integration

State Management: Local state for budget values

Key Features:

- Multi-currency input support
- Real-time validation
- Budget constraint enforcement

2. CurrencySelector Component

Purpose: International currency selection

Dependencies: Currency codes library

State Management: Global currency context

Key Features:

- 150+ international currencies
- Real-time exchange rates
- Localized currency formatting

3. ProductResults Component

Purpose: Display search results from multiple stores

Dependencies: API integrator, affiliate manager

State Management: React Query for data fetching

Key Features:

- Multi-store result aggregation
- Price comparison visualization
- Affiliate link integration

4. AffiliateProduct Component

Purpose: Individual product display with affiliate tracking

Dependencies: Affiliate manager

State Management: Click tracking state

Key Features:

- Commission-optimized display
- Click tracking analytics
- Price history integration

5. ShoppingList Component

Purpose: User's curated product list

Dependencies: Local storage, wishlist integration

State Management: Persistent local state

Key Features:

- Cross-session persistence
- Budget tracking
- Export capabilities

6. Wishlist Component

Purpose: Saved products for future purchase

Dependencies: Product results, local storage

State Management: Persistent wishlist state

Key Features:

- Save/remove functionality
- Price monitoring
- Share capabilities

7. UsageQuota Component

Purpose: Track user API usage and subscription status

Dependencies: Usage API, payment system

State Management: Server-synchronized quota state

Key Features:

- Real-time usage tracking
- Subscription management
- Upgrade prompts

8. AffiliateAnalytics Component

Purpose: Display affiliate performance metrics

Dependencies: Analytics API

State Management: React Query with caching

Key Features:

- Click-through rates
- Commission tracking
- Performance insights

API Architecture

1. Search API (/api/search/route.ts)

Purpose: Aggregate product search across multiple stores

HTTP Methods: POST

Request Schema:

```
{
  query: string;
  budget?: number;
  currency?: string;
  location?: string;
}
```

Response Schema:

```
{
  products: Product[];
  stores: string[];
  totalResults: number;
  searchTime: number;
}
```

External Integrations:

- Amazon Product Advertising API
- eBay Browse API
- Walmart Open API
- Target Redsky API

2. Currency API (/api/currency/route.ts)

Purpose: Real-time currency conversion

HTTP Methods: GET, POST

Features:

- 150+ currency support
- Real-time exchange rates
- Historical rate tracking
- Conversion caching

3. Analytics API (/api/analytics/affiliate-click/route.ts)

Purpose: Track affiliate link clicks and conversions

HTTP Methods: POST

Tracking Data:

- Click timestamps
- User sessions
- Conversion rates
- Commission calculations

4. Usage API (/api/usage/route.ts)

Purpose: Monitor user API quotas and subscription status

HTTP Methods: GET, POST, PUT

Quota Management:

- Free tier: 35 searches
- Paid tier: Unlimited
- Usage analytics
- Billing integration

5. Payments API (/api/payments/route.ts)

Purpose: Handle subscription payments via Stripe

HTTP Methods: POST

Payment Features:

- Stripe integration

- Subscription management
- Invoice generation
- Payment history

Data Flow

1. Product Search Flow

User **Input** → BudgetInput Component → Search API →
API Integrator → External Store APIs →
Affiliate Manager → ProductResults Component →
User **Display**

2. Affiliate Click Flow

User **Click** → AffiliateProduct Component →
Affiliate Manager → Analytics API →
External Store → Commission Tracking

3. Currency Conversion Flow

CurrencySelector → Currency API →
Exchange **Rate** Service →
Component Updates →
Price Recalculation

4. Usage Tracking Flow

API Request → **Usage** Middleware →
Usage API → Quota **Check** →
Component Updates →
Billing Integration

Affiliate System

Architecture Overview

The affiliate system is built around two core modules:

1. AffiliateManager (/lib/affiliate-manager.ts)

Purpose: Central hub for affiliate link generation and commission tracking

Key Features:

- Multi-store affiliate link generation
- Commission rate optimization
- UTM parameter management
- Click tracking integration

Core Methods:

generateAffiliateLink(store: **string**, productId: **string**): **string**
trackClick(affiliateId: **string**, productId: **string**): **void**
calculateCommission(store: **string**, price: **number**): **number**
getTopPerformers(): AffiliateStat[]

2. APIIntegrator (/lib/api-integrator.ts)

Purpose: External API integration with affiliate link injection
Supported Stores:

- **Amazon:** Product Advertising API 5.0
- **eBay:** Browse API v1
- **Walmart:** Open API v3
- **Target:** Redsky API v1

Integration Features:

- Real-time product search
- Price comparison
- Inventory checking
- Affiliate link injection

Commission Structure

Store	Commission Rate	Cookie Duration	Special Features
Amazon	1-10%	24 hours	Prime integration
eBay	1-6%	24 hours	Auction support
Walmart	1-4%	30 days	Grocery eligible
Target	1-8%	7 days	RedCard rewards

Analytics & Tracking

- **Click Tracking:** Real-time affiliate click monitoring
- **Conversion Tracking:** Purchase completion tracking
- **Performance Analytics:** ROI and conversion rate analysis
- **Commission Reporting:** Automated payout calculations

Security Considerations

1. API Security

- **Rate Limiting:** Prevent API abuse
- **Authentication:** Secure API access
- **Input Validation:** Sanitize all user inputs
- **HTTPS Enforcement:** All communications encrypted

2. Data Protection

- **PII Handling:** Minimal personal data collection
- **Session Security:** Secure session management
- **Payment Security:** PCI DSS compliance via Stripe

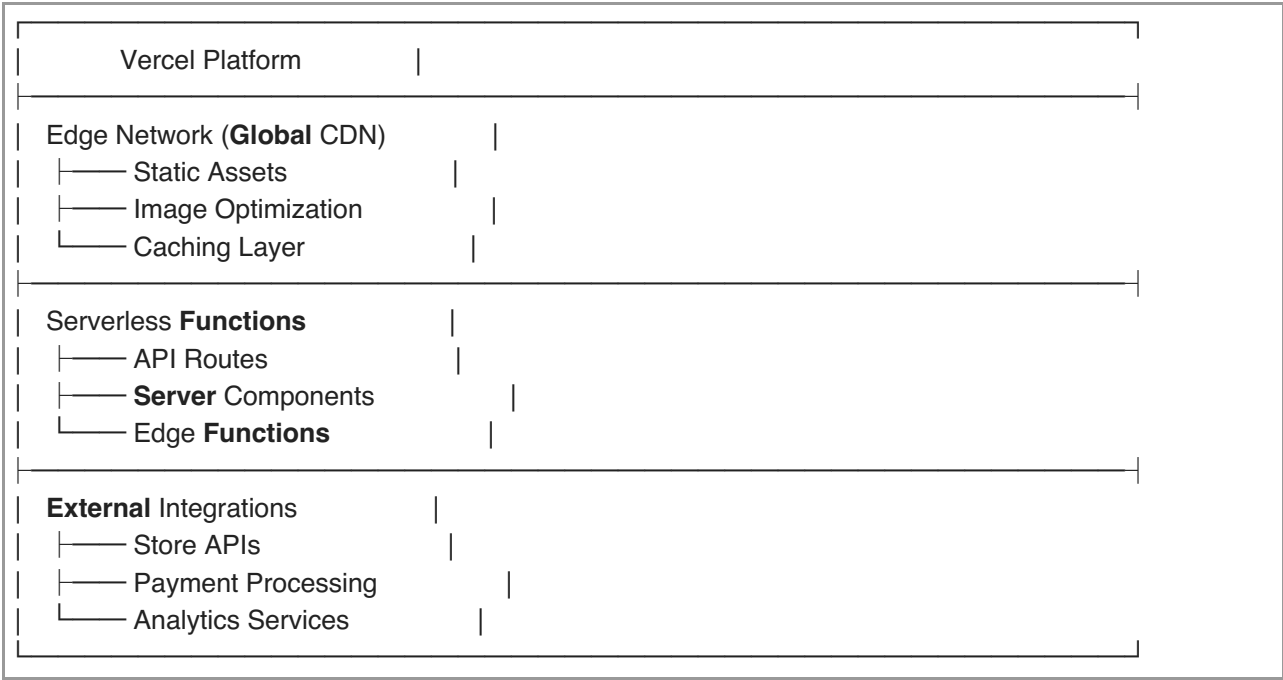
- **Affiliate Data:** Secure commission tracking

3. External API Security

- **API Key Management:** Environment-based key storage
- **Request Signing:** Secure API communications
- **Error Handling:** No sensitive data in error messages
- **Monitoring:** API usage and anomaly detection

Deployment Architecture

Production Environment



Environment Configuration

```
# Store API Keys
AMAZON_ACCESS_KEY=your_amazon_access_key
AMAZON_SECRET_KEY=your_amazon_secret_key
AMAZON_ASSOCIATE_TAG=your_associate_tag
EBAY_APP_ID=your_ebay_app_id
WALMART_API_KEY=your_walmart_api_key
TARGET_API_KEY=your_target_api_key

# Payment Processing
STRIPE_SECRET_KEY=your_stripe_secret_key
STRIPE_PUBLISHABLE_KEY=your_stripe_publishable_key

# Analytics
ANALYTICS_API_KEY=your_analytics_key

# Application
NEXT_PUBLIC_APP_URL=https://smartfete.com
```

Deployment Process

1. **Build Process:** Next.js production build
2. **Static Generation:** Pre-rendered pages
3. **Function Deployment:** Serverless API routes
4. **CDN Distribution:** Global asset distribution
5. **Domain Configuration:** Custom domain setup
6. **SSL Certificate:** Automatic HTTPS

Performance Optimization

1. Frontend Optimization

- **Code Splitting:** Dynamic imports for components
- **Image Optimization:** Next.js Image component
- **Bundle Analysis:** Webpack bundle optimization
- **Lazy Loading:** Component-level lazy loading

2. API Optimization

- **Caching Strategy:** Redis for API response caching
- **Request Batching:** Multiple store search optimization
- **Response Compression:** Gzip compression
- **Connection Pooling:** Efficient database connections

3. External API Optimization

- **Rate Limit Management:** Intelligent request throttling
- **Response Caching:** Store-specific cache strategies
- **Fallback Mechanisms:** Graceful degradation
- **Parallel Processing:** Concurrent API calls

4. Database Optimization

- **Query Optimization:** Efficient data retrieval
 - **Indexing Strategy:** Performance-critical indexes
 - **Connection Management:** Pool configuration
 - **Data Archiving:** Historical data management
-

Future Roadmap

Phase 1: Enhanced Features (Q2 2024)

- **Advanced AI Recommendations:** Machine learning integration
- **Voice Search:** Speech-to-text search capabilities
- **Mobile App:** React Native mobile application
- **Social Features:** Product sharing and reviews

Phase 2: Market Expansion (Q3 2024)

- **Additional Stores:** Best Buy, Home Depot, Costco
- **International Markets:** European and Asian stores
- **Cryptocurrency Payments:** Bitcoin and Ethereum support
- **B2B Features:** Business account management

Phase 3: Advanced Analytics (Q4 2024)

- **Predictive Analytics:** Price forecasting
- **User Behavior Analysis:** Advanced user insights
- **Market Intelligence:** Trend analysis and reporting
- **API Marketplace:** Third-party developer access

Phase 4: Enterprise Solutions (Q1 2025)

- **White-label Solutions:** Custom branding options
 - **Enterprise API:** Bulk search capabilities
 - **Advanced Reporting:** Business intelligence dashboards
 - **Integration Platform:** CRM and ERP integrations
-

Technical Specifications

System Requirements

- **Node.js:** Version 18+ required
- **Memory:** 512MB minimum for serverless functions
- **Storage:** CDN-based static asset storage
- **Bandwidth:** Global CDN distribution

Browser Support

- **Modern Browsers:** Chrome 90+, Firefox 88+, Safari 14+
- **Mobile Browsers:** iOS Safari 14+, Chrome Mobile 90+
- **Accessibility:** WCAG 2.1 AA compliance
- **Performance:** Core Web Vitals optimization

Monitoring & Observability

- **Error Tracking:** Comprehensive error monitoring
 - **Performance Monitoring:** Real-time performance metrics
 - **Uptime Monitoring:** 99.9% availability target
 - **Analytics:** User behavior and conversion tracking
-

Conclusion

SmartFête represents a modern, scalable architecture for AI-powered e-commerce search and affiliate marketing. The system is designed for international scalability, high performance, and comprehensive affiliate revenue optimization.

The modular architecture allows for easy maintenance, feature expansion, and third-party integrations while maintaining security and performance standards required for production e-commerce applications.

Document Version: 1.0

Last Updated: January 2024

Architecture Review Date: Q2 2024